

Tổng Hợp Kiến Thức

I/O là gì?

- **Input/Output (I/O)** = Cách chương trình **"giao tiếp"** với thế giới bên ngoài.
- **Ví dụ:** Đọc/ghi file, mạng (web, API), database.

Đồng bộ (Synchronous) vs. Bất đồng bộ (Asynchronous)

Tính chất	Đồng bộ (Chờ đợi)	Bất đồng bộ (Không cần chờ)
Cách hoạt động	Chương trình dừng lại chờ I/O xong rồi mới làm tiếp.	Chương trình gửi yêu cầu I/O và làm việc khác ngay , khi I/O xong sẽ được thông báo .
Ví dụ	Xếp hàng mua trà sữa (đợi đến lượt, làm xong cho người tiếp theo)	Gọi điện đặt trà sữa (đặt xong đi làm việc khác, khi nào xong người ta gọi lại)

Chặn (Blocking) vs Không chặn (Non-blocking)

Tính chất	Chặn (Blocking)	Không chặn (Non-blocking)
Ảnh hưởng	Làm dừng toàn bộ quá trình làm việc chính.	Không làm dừng quá trình làm việc chính, tiếp tục làm việc khác.
Liên quan I/O	Thường đi với Đồng bộ (Synchronous I/O).	Thường đi với Bất đồng bộ (Asynchronous I/O).
Ví dụ	Đường phố tắc nghẽn (xe phải dừng chờ)	Đường phố thông thoáng (xe đi liên tục)

Kiến trúc Non-blocking của NodeJS = Bí quyết tốc độ

- NodeJS dùng kiến trúc **Bất đồng bộ & Không chặn**.
- **Event Loop (Vòng lặp sự kiện):** "Người quản lý thông minh" của NodeJS:
 1. **Nhận yêu cầu I/O**.
 2. **Giao cho "nhân viên chuyên trách"** (hệ điều hành, thư viện ngoài) xử lý I/O.
 3. **Tiếp tục nhận và xử lý các yêu cầu khác** (không bị chặn).
 4. **Nhận thông báo** khi I/O xong.
 5. **Xử lý kết quả** I/O bằng code JavaScript.
- **Single-threaded (Đơn luồng):** NodeJS chỉ có **1 "nhân viên chính"** (luồng JavaScript), nhưng nhờ Event Loop và "nhân viên chuyên trách" I/O, vẫn **xử lý nhiều việc cùng lúc** hiệu quả.

Lợi ích của Non-blocking I/O trong NodeJS

- **Hiệu suất cao:** Chạy nhanh, xử lý nhiều yêu cầu đồng thời.
- **Mở rộng tốt:** Phục vụ nhiều người dùng (website, ứng dụng web).
- **Tiết kiệm tài nguyên:** Không cần nhiều luồng (threads).
- **Phù hợp ứng dụng web:** Web server, API, ứng dụng thời gian thực (chat, game...).